



Predictive Maintenance Integrated System Hardware and Firmware Research Document

Hardware designing roadmap

1. Reading the Research paper carefully for all the hardware related data.
2. Noting down the number and type of sensors there are and their location on the machine.
3. Selecting the Sensors:
 - a. Looking at the datasheets of the sensors
 - b. Looking at the Research Paper about the hardware.
 - c. Looking at the dataset.
4. Deciding the suitable microcontroller.
5. Designing circuit diagram of the hardware.
6. Designing PCB of the hardware.
 - a. Keep in mind the mechanical aspects.
7. Designing the Case. (Optional)



Table of Contents

Hardware designing roadmap.....	1
Hardware details from Research Paper.....	3
Hardware Design for Real-Time Hydraulic Fault Detection System	3
Computational Architecture as given in paper	4
Hydraulic Fault Component Targets	6
Selecting the Sensors	7
Pressure and Temperature Sensor	7
Motor Power Sensor	7
Vibration Sensor.....	9
Flow Sensor	10
Selecting a microcontroller	11
Designing Hardware	13
System diagram.....	13
Hardware System Diagram.....	13



Hardware details from Research Paper

Hardware Design for Real-Time Hydraulic Fault Detection System

The goal of this hardware design is to implement a real-time fault detection system for hydraulic circuits, as outlined in the referenced study. The system integrates multivariate sensor data acquisition, real-time processing, and robust communication to handle noisy environments and ensure efficient fault detection.

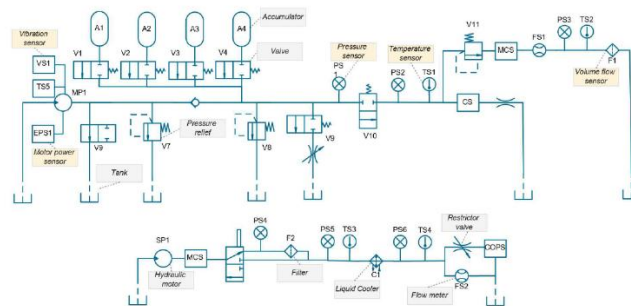


Figure 1. Illustration of the hydraulic system.

The system employs the total of 14 sensors located at different positions on the machine, which are:

1. **Pressure Sensors (PS1–PS6)**
 - Purpose: Measure pressure in hydraulic circuits.
 - Unit: bar.
 - Max. value: 191.92
 - Min. value: 0.0
 - Sampling Rate: 100 Hz -> 6000 readings in 60 seconds.
2. **Motor Power Sensor (EPS1)**
 - Purpose: Monitor motor power.
 - Unit: watts.
 - Max. value: 2995.2
 - Min. value: 2097.8
 - If we consider voltage to be 230VAC then:
 1. Max. current: 13.02 A
 2. Min. current: 9.12 A
 - Sampling Rate: 100 Hz -> 6000 readings in 60 seconds
3. **Volume Flow Sensors (FS1–FS2)**
 - Purpose: Measure volume flow.
 - Unit: liters/minute.
 - Max. value: 20.479
 - Min. value: 0.0
 - Sampling Rate: 10 Hz -> 600 readings in 60 seconds
4. **Temperature Sensors (TS1–TS4)**
 - Purpose: Monitor fluid temperature.
 - Unit: °C.
 - Max. value: 62.176
 - Min. value: 30.355
 - Sampling Rate: 1 Hz -> 60 readings in 60 seconds
5. **Vibration Sensor (VS1)**
 - Purpose: Detect system vibrations.



- Unit: mm/s.
- Max. value: 2.546
- Min. value: 0.483
- Sampling Rate: 1 Hz -> 60 readings in 60 seconds

6. Virtual Sensors

- **Cooling Efficiency Sensor (CE):** Measures cooling efficiency (%)
- **Cooling Power Sensor (CP):** Measures cooling power (kW)
- **Efficiency Factor Sensor (SE):** Evaluates system efficiency (%)

Data Acquisition Rates and Capacities

- Sampling Rates: Between 1 Hz and 100 Hz.
- Packet Size per Sensor: Varies based on the sensor; examples include 47.02 KB for 100 Hz sensors, 4.84 KB for 10Hz sensors and 0.62 KB for 1 Hz sensors.
- Total Data Capacity: The system handles multivariate time-series data collected over **60-second cycles**.

Computational Architecture as given in paper

Edge Computing System

- **Purpose:** The edge computing system is the cornerstone of real-time fault detection. It processes sensor data near the data source to minimize latency and transmission costs, ensuring faster responses and reduced reliance on cloud infrastructure for critical decision-making.
- **Key Features:**
 1. **Real-Time Processing:** The system supports immediate analysis of multivariate time-series data, leveraging pre-trained models to detect faults.
 2. **Noise Handling:** Designed to manage noisy data with robust algorithms, enhancing the accuracy of fault detection.
 3. **Local Decision-Making:** Enables localized operations to continue even during network interruptions.
- **Hardware Specifications:**
 - **Processor:** Intel i5 or equivalent, providing sufficient computational power for running machine learning models and preprocessing tasks.
 - **RAM:** 4 GB to handle high-speed data acquisition and processing.
 - **Storage:** SSD with a capacity sufficient to store transient data and model files.
 - **Operating System:** Windows 10 or Linux, optimized for edge deployment.
- **Advantages:**
 1. **Low Latency:** Near-instantaneous data processing ensures timely fault detection and action.
 2. **Scalability:** Additional edge nodes can be deployed for larger or distributed hydraulic systems.



3. **Cost-Effective:** Reduces data transmission to the cloud, saving bandwidth and lowering operational costs.

Cloud Computing System

- **Purpose:** The cloud computing system complements the edge by providing robust computational capabilities for offline tasks such as feature selection, model training, and large-scale data analysis. It acts as a centralized hub for managing historical data and optimizing system-wide operations.
- **Key Features:**
 1. **Offline Learning:** Performs resource-intensive tasks like genetic algorithm-based feature selection and LSTM model training.
 2. **Data Storage:** Maintains a repository of sensor data for historical analysis and model improvements.
 3. **Scalability:** Supports dynamic resource allocation based on computational demand.
- **Hardware Specifications:**
 - **Processor:** High-performance virtualized or physical servers, capable of handling parallel computational tasks.
 - **RAM:** Typically, 8–16 GB or more, depending on the workload.
 - **Storage:** Network-attached storage (NAS) or SSDs for fast access to large datasets.
 - **Operating System:** Linux-based, optimized for high-throughput computing.
- **Advantages:**
 1. **High Computational Power:** Capable of performing complex analyses that cannot be executed on edge devices.
 2. **Centralized Management:** Provides a single point for model updates and large-scale data monitoring.
 3. **Redundancy:** Ensures system reliability with backup and failover mechanisms.

Communication Between Edge and Cloud

- **Protocol:** Lightweight and efficient communication protocols like MQTT or HTTP/2 are used to transfer data between edge and cloud systems.
- **Bandwidth Optimization:** Data compression and feature selection ensure reduced packet sizes, optimizing bandwidth usage while maintaining critical information.
- **Latency Management:** Prioritization of real-time fault data ensures low-latency responses, while historical data can be transmitted during low-usage periods.
- **Security:** Secure communication protocols (e.g., TLS/SSL) are implemented to protect data integrity and confidentiality during transmission.



Hydraulic Fault Component Targets

The hydraulic system targets multiple fault components. Each component's fault conditions and corresponding operational states are described below:

Cooler Faults

- **Fault Condition:** Cooling power reduction.
- **Control Parameter:** Fan duty cycle.
- **States:**
 1. **100% Efficiency:** Fully operational cooler.
 2. **20% Efficiency:** Reduced efficiency with noticeable degradation.
 3. **3% Efficiency:** Near-total failure.
- **Operational Impact:** A reduction in cooling efficiency leads to overheating, potentially causing system failures.

Valve Faults

- **Fault Condition:** Switching degradation.
- **Control Parameter:** Switchable bypass orifices.
- **States:**
 1. **100% Optimal Behavior:** Fully operational.
 2. **90% Small Lag:** Minimal performance issues.
 3. **80% Severe Lag:** Noticeable delays in response.
 4. **73% Close to Failure:** Near-total loss of functionality.
- **Operational Impact:** Reduced valve responsiveness impacts fluid control, leading to inefficiencies and potential malfunctions.

Pump Faults

- **Fault Condition:** Internal leakage.
- **Control Parameter:** Control current adjustments.
- **States:**
 1. **No Leakage (0):** Fully functional.
 2. **Weak Leakage (1):** Minor efficiency losses.
 3. **Severe Leakage (2):** Significant operational degradation.
- **Operational Impact:** Leakage in the pump reduces system pressure, decreasing overall performance.

Accumulator Faults

- **Fault Condition:** Gas leakage.
- **Control Parameter:** Pre-charge pressure adjustments.
- **States:**
 1. **130 bar:** Optimal pressure.
 2. **115 bar:** Slight reduction in pressure.
 3. **100 bar:** Severe reduction.
 4. **90 bar:** Near-total failure.
- **Operational Impact:** Insufficient gas pressure disrupts energy storage and release, leading to reduced hydraulic efficiency.



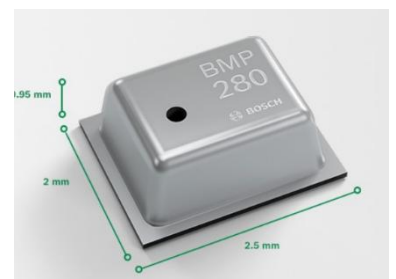
Selecting the Sensors

Pressure and Temperature Sensor

Bosch's BMP280 Environment Sensor

Datasheet: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bmp280-ds001.pdf>

The Bosch BMP280 is a compact, high-performance sensor that measures pressure, temperature, and humidity. It is widely used in various applications due to its accuracy, reliability, and energy efficiency. Below are its specifications and reasons for using it for pressure and temperature measurements:



1. **Pressure Measurement:**
 - **Range:** 300 hPa to 1100 hPa (suitable for altitudes up to 9,000 meters above sea level).
 - **Accuracy:** ± 1 hPa.
 - **Resolution:** 0.18 Pa.
 - **Response Time:** 1.2 ms.
2. **Temperature Measurement:**
 - **Range:** -40°C to $+85^{\circ}\text{C}$.
 - **Accuracy:** $\pm 1.0^{\circ}\text{C}$.
 - **Resolution:** 0.01°C .
3. **Interface:** SPI and I²C compatible, allowing flexible integration with microcontrollers and embedded systems.
4. **Max. Sampling rate:** 157 Hz
5. **Operating Voltage:** 1.71V to 3.6V.
6. **Power Consumption:**
 - Ultra-low power consumption.
 - $\sim 2.7 \mu\text{A}$ in sleep mode.
7. **Physical Dimensions:** Compact package size of $2.5 \times 2.5 \times 0.93$ mm.

Motor Power Sensor

For measuring the power of an AC Motor, we need to measure two components, Current and Voltage. The power consumed by an electrical device can be calculated using the fundamental relationship:

$$P = V \times I \times \cos(\phi)$$

Where:

- P: Real power (in watts).
- V: RMS voltage (in volts).
- I: RMS current (in amperes).
- $\cos(\phi)$: Power factor, representing the phase difference between voltage and current.

Steps to Measure Power



1. **Measure Current:**
 - The SCT-013-030 outputs a voltage proportional to the current flowing through the wire it clamps onto. This output voltage is sampled by an ADC to determine the RMS current.
2. **Measure Voltage:**
 - Use a separate voltage sensor or a potential divider to measure the RMS voltage from the mains.
3. **Calculate Apparent Power:**
 - Apparent Power (S) is the product of RMS voltage and RMS current: $S=V \times I$
4. **Determine Power Factor:**
 - Measure the phase difference (ϕ) between the voltage and current waveforms. This can be done using zero-crossing detection or phase-locked loop techniques.
5. **Calculate Real Power:**
 - Multiply the apparent power by the power factor: $P=S \times \cos(\phi)$
6. **Energy Consumption:**
 - Energy consumed over time is the integral of real power: $E=\int P \, dt$

To know more visit: <https://docs.openenergymonitor.org/electricity-monitoring/ac-power-theory/introduction.html>

SCT-013-030 Current Sensor

The SCT-013-030 Non-Invasive AC Current Sensor is a clamp-style current transformer designed to measure alternating current (AC) without the need for direct electrical connections. This makes it an ideal choice for applications where safety, ease of use, and non-intrusiveness are important. Below are its specifications and advantages, along with reasons for choosing this sensor for your motor power monitoring application.



1. **Current Measurement:**
 - Rated Current: Up to 30A AC.
 - Output Signal: Voltage proportional to the measured current (1V at 30A).
2. **Output Type:**
 - Voltage output via a 3.5mm jack plug.
3. **Frequency Range:**
 - 50 Hz to 1 kHz.
4. **Measurement Accuracy:**
 - Provides accurate current measurement for industrial loads.
5. **Safety:**
 - Fully insulated and safe for non-contact measurement of AC currents.
6. **Physical Design:**
 - Clamp-style design for easy installation and removal without interrupting the circuit.



ZMPT101B AC Single Phase Voltage Sensor

ZMPT101B is a basic but highly useful module that reduces an input voltage by a factor of 5 using a potential divider. The 0-25V Voltage Sensor Module allows you to use a microcontroller's analog input to monitor voltages far higher than it can sense.

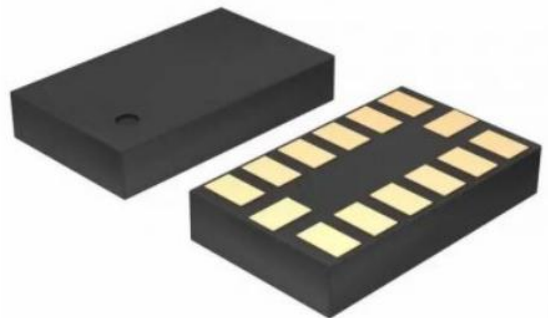


1. **Voltage Measurement:**
 - Maximum Voltage: Up to 250V AC.
 - Output Signal: 0–5V proportional to the input voltage.
2. **Voltage Transformer:**
 - On-board precision micro-voltage transformer for accurate signal transformation and isolation.
3. **Operational Amplifier:**
 - High-precision on-board op-amp circuit for signal sampling, adjustment, and compensation.
4. **Supply Voltage:**
 - Input Range: 5V to 30V DC.
5. **Output Mode:**
 - Adjustable output mode for compatibility with various applications.
6. **Operating Temperature:**
 - Range: -40°C to +70°C.
7. **Physical Features:**
 - Compact, lightweight design for easy PCB integration.

Vibration Sensor

ADXL345 Accelerometer Sensor

The **ADXL345** is a low-power, high-precision, 3-axis accelerometer designed to measure acceleration in applications requiring low power consumption and high performance. It is widely used in embedded systems, portable devices, and IoT applications for detecting motion, orientation, and vibration.



1. **Measurement Range:**
 - $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$ (selectable).
2. **Resolution:**
 - 13-bit resolution with a sensitivity of 4 mg/LSB at $\pm 2g$.
3. **Power Supply:** Operating Voltage: 2.0V to 3.6V.
4. **Interfaces:**
 - **I²C:** Up to 400 kHz.
 - **SPI:** Up to 10 MHz.
5. **Power Consumption:**
 - 40 μA in measurement mode.
 - 0.1 μA in standby mode.
6. **Output Data Rate:** 0.1 Hz to 3200 Hz.
7. **Operating Temperature:** -40°C to +85°C.
8. **Package:** Compact 3 mm × 5 mm × 1 mm LGA package



Flow Sensor

Sea YFS201 Flowmeter

The **YFS201** is a water flow sensor designed for measuring flow rates in water systems. It operates on a turbine and Hall-effect principle to generate pulse signals proportional to the flow rate. The sensor is ideal for applications such as solar energy pumps, gas water heaters, intelligent water meters, and other water control systems.



1. **Flow Range:**
 - 1 to 30 L/min.
2. **Voltage and Current:**
 - Operating Voltage: 5V to 18V DC.
 - Current Consumption: $\leq 15\text{mA}$ @5VDC.
3. **Pulse Output:**
 - **Flow Rate Formula:** $F = 7.5 * Q$ F=Frequency (Hz), Q=Flow Rate (L/min)
 - Pulse Duty Cycle: 50%.
 - Cumulative Flow Conversion: 1 Liter of water equals 450 pulses.
 - Accuracy: $\pm 10\%$.
4. **Hydraulic Pressure:**
 - Withstands pressure up to 1.75 MPa.
5. **Electrical Properties:**
 - Insulation Resistance: $> 100\text{ M}\Omega$.
 - Electrical Strength: AC 500V, 50Hz.
6. **Output Signal Levels:**
 - Rated Voltage: 5V DC.
 - High Level: $\geq 4.5\text{V}$.
 - Low Level: $\leq 0.5\text{V}$.
7. **Operating Conditions:**
 - Temperature: -25°C to 80°C .
 - Water Quality: Clean Water.
8. **Size and Diameter:**
 - Diameter: 20 mm.



Selecting a microcontroller

For this system ESP32 microcontroller will be best suitable. The **Espressif ESP32-S3-WROOM-1-N4R8** microcontroller is a powerful, versatile, and cost-effective choice for this project. Its robust features, high processing speed, and integrated communication capabilities make it ideal for handling multiple sensors and performing real-time data acquisition and processing.



Key Features of ESP32

1. **Processing Power:**
 - **Dual-Core Xtensa LX6 Processor:**
 - Clock Speed: Up to 240 MHz, significantly faster than many traditional microcontrollers like the ATmega2560.
 - **Multitasking:**
 - Capable of running real-time sensor data acquisition while simultaneously handling data transmission, signal processing, or user interface tasks.
2. **Memory:**
 - **RAM:** 520 KB SRAM, allowing sufficient space for buffers and data processing.
 - **Flash Storage:** 4 MB or more (depending on the module), perfect for storing firmware, calibration data, and libraries.
3. **Communication Interfaces:**
 - **I²C:** Two hardware I²C buses for interfacing with sensors like **BME280** and **ADXL345**.
 - **SPI:** Multiple SPI interfaces for high-speed communication with sensors like **ADXL345** if I²C is not preferred.
 - **ADC:** 18-channel 12-bit ADC, offering higher precision for analog sensors like **SCT-013-030** and **ZMPT101B**.
 - **GPIOs:** ~34 GPIOs, configurable for digital, analog, or communication protocols.
 - **UART:** Multiple UART ports for debugging or additional peripherals.
4. **Built-In Connectivity:**
 - **Wi-Fi and Bluetooth:** Enables IoT features for remote monitoring and control for future purpose.
5. **Power Efficiency:**
 - Supports low-power modes for energy-efficient operations, suitable for battery-powered systems.
6. **Compact Size:**
 - Small form factor ideal for embedded systems with space constraints.



Pinout of ESP32

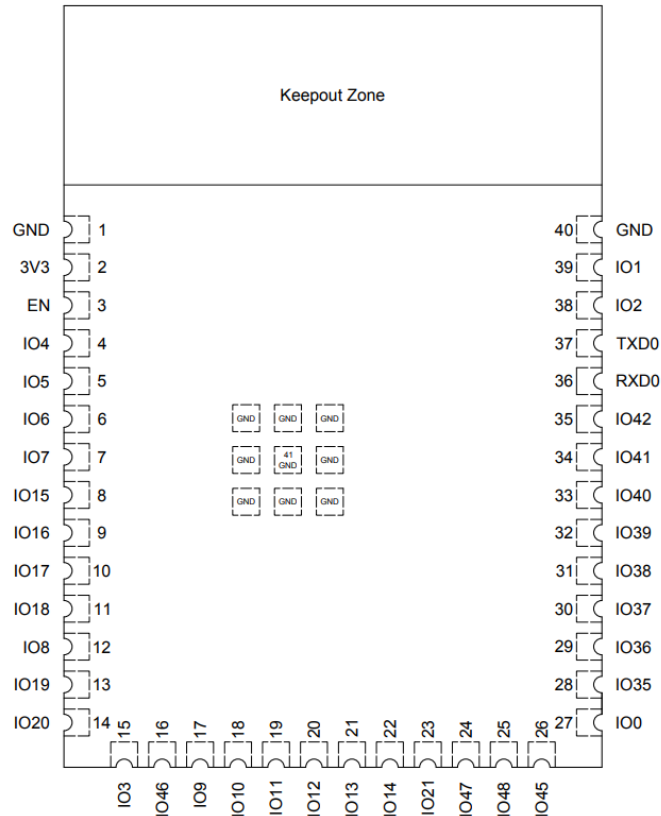


Figure 3: Pin Layout (Top View)

Table 3: Pin Definitions

Name	No.	Type ^a	Function
GND	1	P	GND
3V3	2	P	Power supply
EN	3	I	High: on, enables the chip. Low: off, the chip powers off. Note: Do not leave the EN pin floating.
IO4	4	I/O/T	RTC_GPIO4, GPIO4 , TOUCH4, ADC1_CH3
IO5	5	I/O/T	RTC_GPIO5, GPIO5 , TOUCH5, ADC1_CH4
IO6	6	I/O/T	RTC_GPIO6, GPIO6 , TOUCH6, ADC1_CH5
IO7	7	I/O/T	RTC_GPIO7, GPIO7 , TOUCH7, ADC1_CH6
IO15	8	I/O/T	RTC_GPIO15, GPIO15 , U0RTS, ADC2_CH4, XTAL_32K_P
IO16	9	I/O/T	RTC_GPIO16, GPIO16 , U0CTS, ADC2_CH5, XTAL_32K_N
IO17	10	I/O/T	RTC_GPIO17, GPIO17 , U1TXD, ADC2_CH6
IO18	11	I/O/T	RTC_GPIO18, GPIO18 , U1RXD, ADC2_CH7, CLK_OUT3
IO8	12	I/O/T	RTC_GPIO8, GPIO8 , TOUCH8, ADC1_CH7, SUBSPICS1
IO19	13	I/O/T	RTC_GPIO19, GPIO19 , U1RTS, ADC2_CH8, CLK_OUT2, USB_D-
IO20	14	I/O/T	RTC_GPIO20, GPIO20 , U1CTS, ADC2_CH9, CLK_OUT1, USB_D+
IO3	15	I/O/T	RTC_GPIO3, GPIO3 , TOUCH3, ADC1_CH2
IO46	16	I/O/T	GPIO46
IO9	17	I/O/T	RTC_GPIO9, GPIO9 , TOUCH9, ADC1_CH8, FSPICHD, SUBSPIHD
IO10	18	I/O/T	RTC_GPIO10, GPIO10 , TOUCH10, ADC1_CH9, FSPICS0, FSPIC04, SUBSPICS0
IO11	19	I/O/T	RTC_GPIO11, GPIO11 , TOUCH11, ADC2_CH0, FSPID, FSPIC05, SUBSPID
IO12	20	I/O/T	RTC_GPIO12, GPIO12 , TOUCH12, ADC2_CH1, FSPICLK, FSPIC06, SUBSPICLK
IO13	21	I/O/T	RTC_GPIO13, GPIO13 , TOUCH13, ADC2_CH2, FSPIQ, FSPIC07, SUBSPIQ
IO14	22	I/O/T	RTC_GPIO14, GPIO14 , TOUCH14, ADC2_CH3, FSPWP, FSPIDQS, SUBSPIWP
IO21	23	I/O/T	RTC_GPIO21, GPIO21
IO47 ^c	24	I/O/T	SPICLK_P_DIFF, GPIO47 , SUBSPICLK_P_DIFF
IO48 ^c	25	I/O/T	SPICLK_N_DIFF, GPIO48 , SUBSPICLK_N_DIFF
IO45	26	I/O/T	GPIO45
IO0	27	I/O/T	RTC_GPIO0, GPIO0
IO35 ^b	28	I/O/T	FSPIC06, GPIO35 , FSPID, SUBSPID
IO36 ^b	29	I/O/T	FSPIC07, GPIO36 , FSPICLK, SUBSPICLK

Name	No.	Type ^a	Function
IO37 ^b	30	I/O/T	SPIDQS, GPIO37 , FSPIQ, SUBSPIQ
IO38	31	I/O/T	GPIO38 , FSPWP, SUBSPIWP
IO39	32	I/O/T	MTCK , GPIO39, CLK_OUT3, SUBSPICS1
IO40	33	I/O/T	MTDO , GPIO40, CLK_OUT2
IO41	34	I/O/T	MTDI , GPIO41, CLK_OUT1
IO42	35	I/O/T	MTMS , GPIO42
RXD0	36	I/O/T	U0RXD , GPIO44, CLK_OUT2
TXD0	37	I/O/T	U0TXD , GPIO43, CLK_OUT1
IO2	38	I/O/T	RTC_GPIO2, GPIO2 , TOUCH2, ADC1_CH1
IO1	39	I/O/T	RTC_GPIO1, GPIO1 , TOUCH1, ADC1_CH0
GND	40	P	GND
EPAD	41	P	GND

^a P: power supply; I: input; O: output; T: high impedance. Pin functions in bold font are the default pin functions. For pin 28 ~ 30, the default function is decided by eFuse bit.

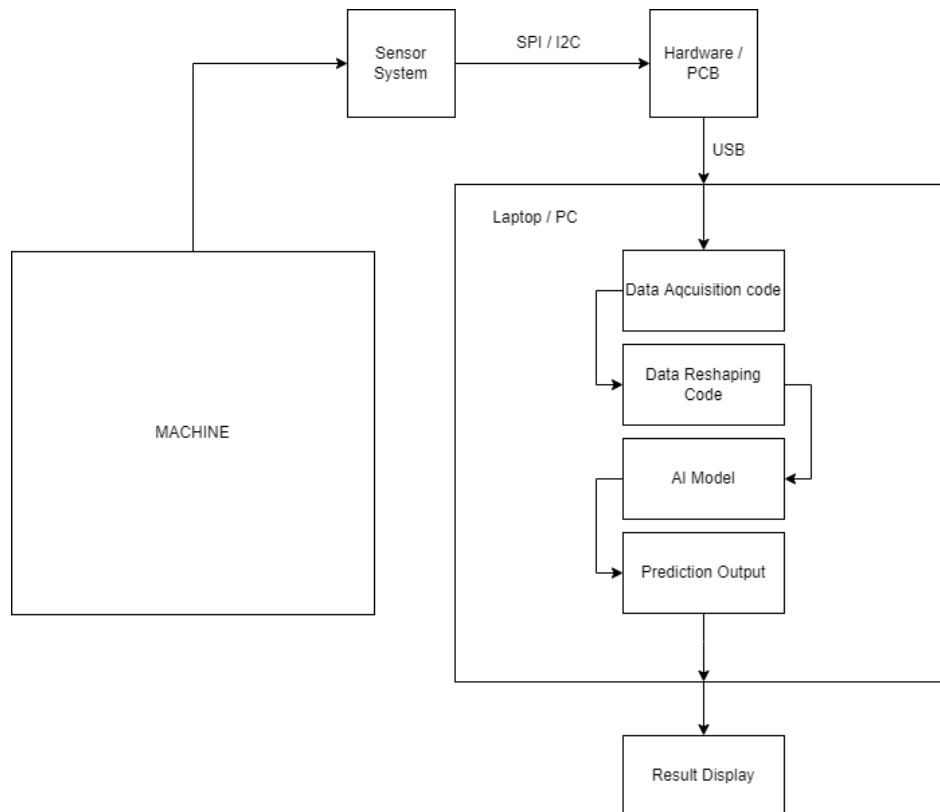
^b For modules with Octal SPI PSRAM, i.e., modules embedded with ESP32-S3R8 or ESP32-S3R16V, pins IO35, IO36, and IO37 are connected to the Octal SPI PSRAM and are not available for other uses.

^c For modules embedded with ESP32-S3R16V, as the VDD_SPI voltage of the ESP32-S3R16V chip is set to 1.8 V, the working voltage for GPIO47 and GPIO48 is also 1.8 V, which is different from other GPIOs.

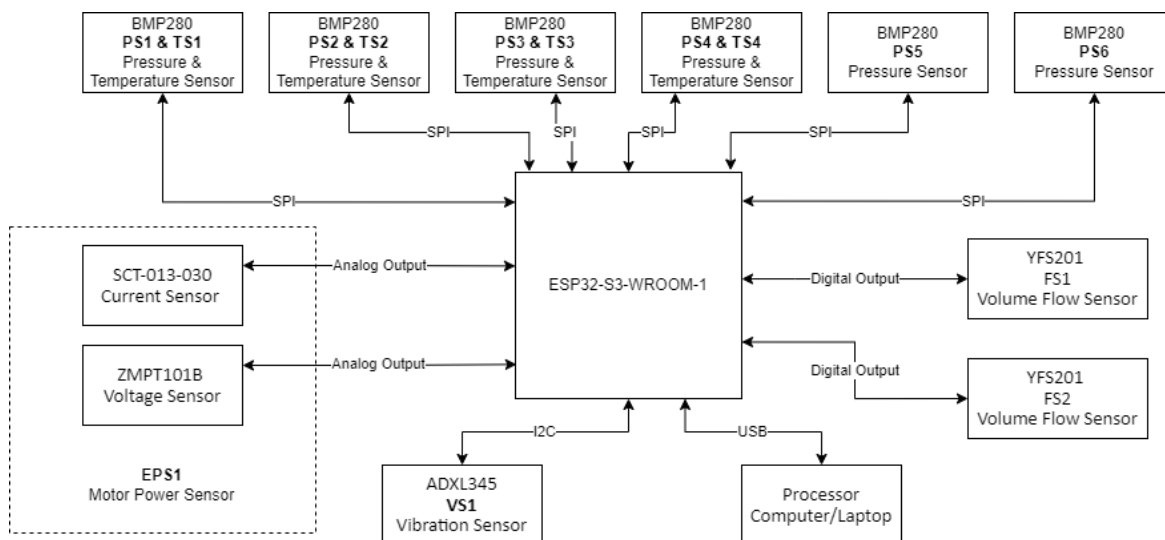


Designing Hardware

System diagram



Hardware System Diagram





Component Current Consumption Calculation

Component Name	Max. Current Consumption (mA)	Qty.	Total Consumption (mA)
ESP32-S3-WROOM-1	240	1	240
BMP280	2	6	12
SCT-013-030	0	1	0
ZMPT101B	3	1	3
YFS201	15	2	30
ADXL345	0.5	1	0.5
	TOTAL		~285.5

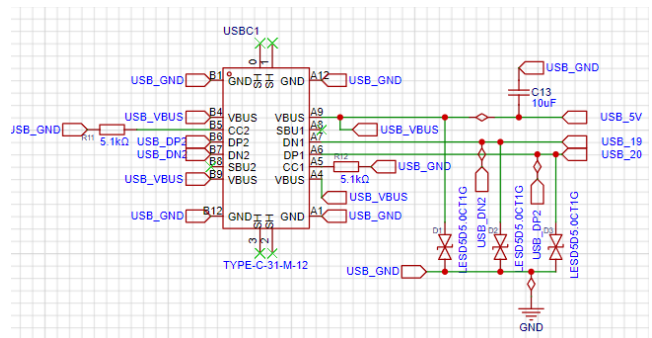
Designing Programming Circuit:

This section will explain the circuitry needed for flashing the firmware and powering the microcontroller. Chose CDC (Communication Device Class) over USB-to-Serial for several reasons:

- **Flexibility:** CDC allows the ESP32-S3 to be recognized as a virtual COM port, making it more versatile.
- **No extra drivers:** Most operating systems natively support CDC, so no need for USB-to-Serial drivers.
- **Better performance:** CDC offers higher data transfer speeds, crucial for real-time telemetry.
- **Simpler setup:** CDC avoids extra hardware like USB-to-UART converters, reducing complexity and PCB size.

In CDC, the ESP32-S3 uses its USB OTG feature to act as a virtual serial port. Data is sent over USB, handled by a driver, and treated like standard serial communication. The GPIO19 (USB D+) and GPIO20 (USB D-) can directly be connected to the ESP32.

The precautionary circuit in a USB-C setup using three **LES5D5.0CT1G** components is designed to protect against electrostatic discharge (ESD) and transient voltage surges that could damage sensitive circuitry.



LES5D5.0CT1G (Low-Capacitance ESD Protection Diodes):

These components are **ESD protection diodes** specifically designed to protect USB lines from voltage spikes or electrostatic discharge events. Each LES5D5.0CT1G is rated for a breakdown voltage of 5V, which is suitable for the data and power lines in USB-C.

Why Use Three Diodes?

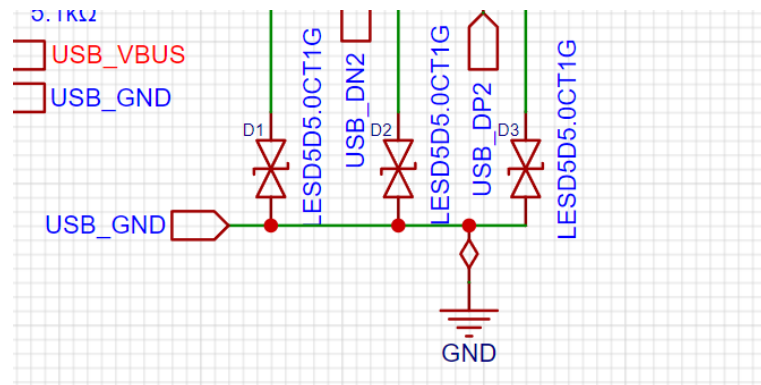
The USB-C connection typically needs to protect multiple lines—such as the power, ground, and data (D+/D-) lines—so the three LES5D5.0CT1G diodes are placed on key signal lines. Here is how they typically work:

1. **Data Lines (D+ and D-):** Two diodes are placed across these lines to prevent ESD or voltage surges from damaging the sensitive ICs that process USB signals.
2. **Vbus Line:** The third diode is used to protect the 5V (Vbus) line from any transient voltage spikes.



How It Works:

- **Normal Operation:** Under normal operating conditions, these diodes are inactive, allowing data and power to flow without interference.
- **ESD Event:** If a high-voltage ESD event occurs (e.g., static discharge when plugging in a device), the diodes instantly conduct and divert the high voltage away from the sensitive circuit to ground, preventing damage.



Designing Power Supply

As mentioned in the above topic “Component Current Consumption Calculation” the maximum current system will need is around 285.5 mA and with a safety margin 400-500 mA, so there is a need of 1000mA power supply.

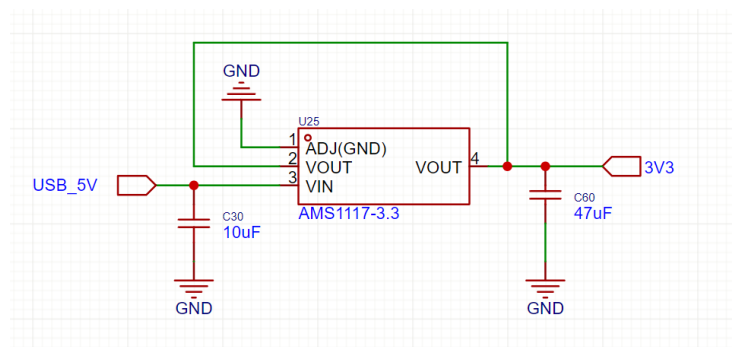
When the USB will be connected to the PCB it will supply 5V DC but majorly in this system 3.3v is require. This 5V from USB can be step down using a step-down voltage converter such as AMS1117-3.3



The **AMS1117-3.3** is a linear voltage regulator designed to provide a stable 3.3V output from a higher input voltage. It is widely used in embedded systems and IoT applications to power microcontrollers, sensors, and other components requiring a precise 3.3V supply. Its simplicity, reliability, and ease of integration make it a popular choice for voltage regulation in low-power applications.

AMS1117-3.3 Specifications

1. **Output Voltage:** Fixed at **3.3V**.
2. **Input Voltage Range:** **4.5V to 15V**.
3. **Output Current:** Maximum: **1A**.
4. **Dropout Voltage:**
 - **1.1V (Typical)** at 1A load.
 - Requires input voltage to be at least **4.4V** for proper operation.
5. **Efficiency:**
 - Linear regulator; efficiency depends on the input-output voltage difference.





- Less efficient than switch-mode power supplies for high input-output voltage differences.
6. **Output Voltage Tolerance:** $\pm 1\%$ at 25°C .
 7. **Thermal Protection:** Built-in protection against overheating.
 8. **Short-Circuit Protection:** Safeguards against damage due to excessive load.

For filtering out any fluctuation capacitors are needed which are shown in the circuit diagram.

Designing Sensor interface

Pressure/Temperature Sensor:

The six pressure and temperature sensors (PS1-PS6) are interfaced with the ESP32-S3 microcontroller using the SPI (Serial Peripheral Interface) protocol. The ESP32-S3 acts as the SPI master, while the sensors function as SPI slaves.

Each sensor has four main SPI lines:

- MISO (Master In Slave Out) – Connected to ESP32-S3 MISO (receives data from sensors).
- MOSI (Master Out Slave In) – Connected to ESP32-S3 MOSI (sends commands to sensors).
- SCK (Serial Clock) – Connected to ESP32-S3 SCK (clock signal for synchronization).
- CS (Chip Select) – Each sensor has a dedicated CS pin, connected to separate GPIO pins on the ESP32-S3 to enable individual sensor communication.

The wiring is as follows:

Sensor	MISO	MOSI	SCK	CS (GPIO)
PS1	GPIO_11	GPIO_13	GPIO_10	GPIO_1
PS2	GPIO_11	GPIO_13	GPIO_10	GPIO_2
PS3	GPIO_11	GPIO_13	GPIO_10	GPIO_41
PS4	GPIO_11	GPIO_13	GPIO_10	GPIO_40
PS5	GPIO_11	GPIO_13	GPIO_10	GPIO_39
PS6	GPIO_11	GPIO_13	GPIO_10	GPIO_38

(Where CS(GPIO) are unique GPIO pins assigned for chip select).



Configure SPI bus on the ESP32-S3, specifying MISO, MOSI, SCK, and multiple CS pins:

```
void setup() {
    Serial.begin(115200);
    spi.begin(SCK_PIN, MISO_PIN, MOSI_PIN); // Initialize SPI

    // Set CS pins as OUTPUT and disable all sensors
    for (int i = 0; i < 6; i++) {
        pinMode(csPins[i], OUTPUT);
        digitalWrite(csPins[i], HIGH);
    }
}
```

Set SPI clock speed, mode, and bit order (as per sensor datasheet). Sensor Selection and Data Exchange:

```
uint16_t readSensorData(int sensorIndex, uint8_t regAddr) {
    uint16_t data = 0;
    digitalWrite(csPins[sensorIndex], LOW); // Select sensor

    spi.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
    spi.transfer(regAddr); // Send register address
    data = spi.transfer(0x00) << 8; // Read MSB
    data |= spi.transfer(0x00); // Read LSB
    spi.endTransaction();

    digitalWrite(csPins[sensorIndex], HIGH); // Deselect sensor
    return data;
}
```

- To communicate with a specific sensor (e.g., PS1), set CS1 LOW (activate), while keeping other CS lines HIGH.
- Send commands via MOSI and receive responses via MISO.
- After reading/writing data, set CS1 HIGH (deactivate).
- Repeat for PS2–PS6 as needed.

Data Processing:

```
for (int i = 0; i < 6; i++) {
    uint16_t pressure = readSensorData(i, 0x01); // Read pressure register
    uint16_t temperature = readSensorData(i, 0x02); // Read temperature register

    Serial.print("PS");
    Serial.print(i + 1);
    Serial.print(" - Pressure: ");
    Serial.print(pressure);
    Serial.print(" | Temperature: ");
    Serial.println(temperature);
}
```

- Read raw pressure and temperature values.
- Convert values based on sensor-specific calibration formulas.



Vibration Sensor interface

The ADXL345 accelerometer communicates via I2C, requiring two pins:

- SDA (Data) → ESP32-S3 GPIO_9
- SCL (Clock) → ESP32-S3 GPIO_10

The ADXL345 has a default I2C address of 0x53. The following code demonstrates initialization, reading acceleration data, and displaying vibration levels.

Include Libraries and Define I2C Pins:

```
#include <Wire.h>

#define ADXL345_ADDR 0x53 // I2C address of ADXL345
#define SDA_PIN      21   // I2C Data
#define SCL_PIN      22   // I2C Clock

void setup() {
  Serial.begin(115200);
  Wire.begin(SDA_PIN, SCL_PIN); // Initialize I2C
  initADXL345();
}
```

Initialize ADXL345:

```
void initADXL345() {
  Wire.beginTransmission(ADXL345_ADDR);
  Wire.write(0x2D); // Power control register
  Wire.write(0x08); // Enable measurement mode
  Wire.endTransmission();

  Wire.beginTransmission(ADXL345_ADDR);
  Wire.write(0x31); // Data format register
  Wire.write(0x0B); // Full resolution, ±16g range
  Wire.endTransmission();
}
```

Read Vibration Data (Acceleration on X, Y, Z):

```
void readAcceleration(int16_t &x, int16_t &y, int16_t &z) {
  Wire.beginTransmission(ADXL345_ADDR);
  Wire.write(0x32); // Start register (X-axis LSB)
  Wire.endTransmission(false);

  Wire.requestFrom(ADXL345_ADDR, 6); // Read 6 bytes (X, Y, Z)

  x = Wire.read() | (Wire.read() << 8);
  y = Wire.read() | (Wire.read() << 8);
  z = Wire.read() | (Wire.read() << 8);
}
```



Current Sensor interface

The SCT-013-030 is a non-invasive AC current sensor with a 30A / 1V output, meaning it provides a voltage proportional to the current flowing through the measured conductor. Since it outputs AC voltage, an analog-to-digital conversion (ADC) with offset biasing is required to read current values accurately.

Since **ESP32-S3 operates at 3.3V**, we need to ensure the **output signal remains within the ADC range (0 - 3.3V)**. The SCT-013-030 requires:

- A **burden resistor (inside the sensor, already present for 30A/1V version)**
- A **voltage divider + capacitor** to center the waveform at **1.65V (midpoint of 0-3.3V range)**

Hardware connections:

Signal Tip -> GPIO_7

Ground Ring -> Voltage Divider circuit

Biasing Circuit (for AC Signal Centering):

To keep the AC signal within **ESP32 ADC range**, we use **two 100kΩ resistors** to create a **1.65V offset**, along with a **10μF capacitor** for smoothing.

Read and convert AC Current Values:

Since **AC voltage oscillates around 1.65V**, we must:

1. **Read multiple samples** to find the **peak-to-peak value**.
2. **Calculate RMS current** using the **conversion formula**.

```
float readCurrent() {
    int samples = 500;
    float sum = 0;

    for (int i = 0; i < samples; i++) {
        int adcValue = analogRead(SENSOR_PIN);
        float voltage = (adcValue / (float)ADC_RES) * V_REF; // Convert ADC to voltage
        float centeredVoltage = voltage - (V_REF / 2); // Remove offset
        sum += centeredVoltage * centeredVoltage; // Square for RMS
        delayMicroseconds(100);
    }

    float meanVoltage = sqrt(sum / samples); // RMS voltage
    float current = meanVoltage * CALIBRATION; // Convert to current using calibration factor

    return current;
}
```